TAREQO

06 — Authentification

L'authentification est la première étape du modèle **SaaS multi-tenant** : chaque commerçant est un *tenant*, et l'identité de l'utilisateur est ce qui sépare les données de chaque magasin (à l'avenir, via Row Level Security sur le Postgres de Supabase).

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Flux général 3

Configuration 3

Flux implémentés 4

- Se connecter (`SignInUseCase` → `signInWithPassword`) 4
- S'inscrire (`SignUpUseCase` → `signUp`) 4
- Se déconnecter (`SignOutUseCase` → `signOut`) 4

Session persistante (comment l'app « se souvient » de l'utilisateur) 5

Approbation d'accès (propriétaire → tenant) 5

Employé désactivé (accès suspendu) 5

Multi-tenant et RLS (pourquoi l'authentification vient en premier) 6

Documents connexes 6

Flux général

```
main.dart
  SupabaseBootstrap.initialize() # Supabase.initialize(url, anonKey)
  ServiceLocator.initialize() # enregistre IAuthRepository → AuthRepositoryImpl → SupabaseAuthRemoteDataSourceImpl
  runApp(ProviderScope(...))
AuthGate (écran racine)
  authStateProvider (StreamProvider<AppUser?>)
  loading → splash (restauration de la session persistée)
  user == null → LoginScreen
  user != null → route par rôle (voir Multi-tenant)  super admin (rôle dans la DB) → AdminShell
  sans tenant → TenantOnboardingScreen (choisir un forfait)
  tenant pending → TenantPendingScreen (≤ 24 h)
  tenant rejected → TenantRejectedScreen (motif)
  tenant approved → AppShell (ERP)
```

La chaîne complète (toute la conversion de l'utilisateur Supabase User en AppUser du domaine se produit dans le datasource) :

```
LoginScreen → SignInUseCase → IAuthRepository → AuthRepositoryImpl → AuthRemoteDataSource →
SupabaseAuthRemoteDataSourceImpl → gotrue (Supabase)
```

Configuration

lib/core/config/supabase_config.dart :

```
static const String url = String.fromEnvironment(
 'SUPABASE_URL', defaultValue: 'https://dotklbpqmlnhcdifiukd.supabase.co',
);
static const String anonKey = String.fromEnvironment(
 'SUPABASE_ANON_KEY', defaultValue: 'eyJ...', // clé anon publique par conception
);
static bool get isConfigured => url.isNotEmpty && anonKey.isNotEmpty;
```

- La clé anon est publique par conception dans Supabase (elle est embarquée dans le client) — la vraie sécurité vient du RLS. Ne mettez jamais la clé *service role* dans le client.
- Flavors (dev/prod) : les identifiants sont fournis via --dart-define-from-file (env/dev.env commité, env/prod.env gitignoré). FLUTTER_APP_FLAVOR est un define réservé injecté depuis --flavor — ne le définissez pas manuellement :

```
flutter run --flavor dev --dart-define-from-file=env/dev.env
flutter build appbundle --flavor prod --release \
 --dart-define-from-file=env/prod.env
```

- Fail-fast en release (S2) : un build release avec les défauts embarqués (ou sans --flavor prod) lève une erreur au démarrage — un APK publié ne peut jamais pointer silencieusement vers le projet dev. En debug, les défauts conviennent au développement.

- Sans identifiants, SupabaseBootstrap.initialize() ne fait que journaliser un avertissement ; l'app s'ouvre et authStateProvider émet null (déconnecté) — la connexion échoue avec un message convivial au lieu de planter.

Flux implémentés

Se connecter (SignInUseCase → signInWithPassword)

1. LoginScreen._submit() valide le formulaire (e-mail avec domaine/TLD, mot de passe ≥ 8 caractères, confirmation identique à l'inscription) ;
2. Appelle signInUseCaseProvider → IAuthRepository.signIn → gotrue ;
3. Succès : le flux authStateChanges émet l'AppUser → AuthGate bascule vers AppShell automatiquement (l'écran ne navigue pas manuellement) ;
4. Échec : log détaillé via debugPrint + SnackBar générique (« Não foi possível entrar. Verifique e-mail e senha. ») — n'expose pas les internals du provider.

S'inscrire (SignUpUseCase → signUp)

1. Formulaire avec nom (facultatif), e-mail et mot de passe ;
2. signUp(email, password, data: {'full_name': ...}) ;
3. Avec la confirmation d'e-mail activée (défaut Supabase), response.user peut être null — le compte est créé mais la session n'arrive qu'après confirmation. L'UI affiche : *« Conta criada! Confirme seu e-mail para entrar. »*
4. Dans ce cas, le datasource renvoie un AppUser avec un id placeholder (fallbackEmail) — aucune donnée ne doit être associée à cet id ; la vraie session arrive après confirmation.
5. Deep link (Aug 8) : le lien de confirmation de l'e-mail pointe vers tareqo://callback (le site_url du projet) — l'OS ouvre l'app, et le DeepLinkHandler (lib/core/network/deep_link_handler.dart, package app_links) transmet l'URI à gotrue (getSessionFromUrl), qui termine la confirmation et le flux de session réagit tout seul. Sans cela, le lien atterrissait sur http://localhost:3000 (défaut) et la confirmation ne se terminait jamais.

> Limite d'envoi d'e-mails (Aug 8) : sur le plan gratuit, quand la quota d'envoi est épuisée (~2/heure), GoTrue répond **429 « email rate limit exceeded »** — le LoginScreen l'attrape (AuthApiException avec statusCode 429) et affiche *« Trop de tentatives consécutives. Attendez environ 1 heure et réessayez. »* (clé loginRateLimitError, pt/en/fr) au lieu de l'erreur générique. C'est transitoire : l'inscription reprend à la fenêtre suivante. Pour la production, voir le plan SMTP personnalisé dans Go-Live §2.6.

Se déconnecter (SignOutUseCase → signOut)

Bouton « Se déconnecter » sur le tableau de bord → signOutUseCaseProvider.call() → le flux émet null → AuthGate revient à la connexion.

Session persistante (comment l'app « se souvient » de l'utilisateur)

1. Stockage : la session est persistée via un `LocalStorage` personnalisé câblé dans `Supabase.initialize` — `SecureSupabaseLocalStorage` le soutient avec `flutter_secure_storage` (Keychain/Keystore) sur les plateformes natives (le défaut du SDK utiliserait shared_preferences en texte clair — corrigé le Aug 6). Le vérificateur de code PKCE utilise le même backend sécurisé. Sur web, le défaut du SDK (window.localStorage) est conservé — flutter_secure_storage 10.x n'a pas de support web (limitation documentée).
2. Restauration au démarrage : onAuthStateChange ré-émet la session enregistrée. Le AuthGate réagit : session restaurée → directement au tableau de bord, pas de nouvelle connexion.
3. Renouvellement automatique : gotrue renouvelle le jeton d'accès avec le jeton de rafraîchissement avant son expiration.
4. Réactivité : connexion/déconnexion émettent des événements sur le même flux ; la navigation et tout état dépendant de la session réagissent automatiquement.

> Attention : Note de migration : les installations avec une session stockée précédemment par le défaut du SDK se déconnecteront une fois après la mise à niveau (la session passe dans le Keychain/Keystore).

Approbation d'accès (propriétaire → tenant)

Après l'inscription, le propriétaire passe par l'onboarding (TenantOnboardingScreen) : choisit le forfait et saisit le nom de l'entreprise → crée la demande (RequestTenantAccessUseCase). Le AuthGate route ensuite selon le statut du tenant : pending (écran d'attente ≤ 24 h), rejected (motif) ou approved (ERP complet). Le super administrateur décide des demandes dans le panneau d'administration — détails dans Multi-tenant et Super Admin.

Employé désactivé (accès suspendu)

Le propriétaire active/désactive les employés dans l'onglet Employés du module Magasins (interrupteur + confirmation ; RPC set_employee_active). Pour un employé désactivé, le blocage est à deux niveaux :

- App : le AuthGate résout les liens de l'utilisateur dans store_employees ; si tous ont is_active = false, il affiche l'écran « Accès suspendu » (_BlockedEmployeeScreen — avec bouton de déconnexion) au lieu de l'ERP. Le blocage est réactif et instantané : en mode Supabase, l'app s'abonne à Realtime (postgres_changes sur store_employees filtré par user_id = auth.uid()) — quand le propriétaire désactive l'employé sur UN AUTRE appareil, l'événement arrive immédiatement et la session tombe sur l'écran suspendu sans redémarrer l'app (la réactivation restaure l'accès par le même flux). Un timer de repli (30 s, employeePollIntervalProvider, centralisé dans employeesPollingProvider) re-récupère les liens en mode local et quand le websocket est coupé — et tout le polling (timer + Realtime) est annulé à la déconnexion (autoDispose), rien ne tourne en arrière-plan ;
- Serveur (autorité) : is_my_store/is_manager_of_store exigent un lien actif — le RLS et les gardes RPC refusent toute opération dans les magasins, même pour un client modifié appelant PostgREST directement.

Détails d'implémentation (la colonne store_employees.is_active, la RPC et les helpers) dans Multi-tenant et Super Admin.

Multi-tenant et RLS (pourquoi l'authentification vient en premier)

1. Isolation des données : sans connexion, impossible de séparer les données de chaque magasin dans la même base ;
2. Sécurité dans la base : Supabase applique le JWT dans Row Level Security — auth.uid() = user_id ou public.is_my_store(store_id) (propriétaire du tenant = tous les magasins ; employé lié = son magasin) — même si l'UI fuit des données, la base les filtre ;
3. Modèle de revenus : l'identité identifie qui paie, quel forfait et quelles quotas s'appliquent ;
4. Onboarding/rétention : la session persistante réduit la friction quotidienne.

> État actuel : l'authentification + la synchronisation SaaS complète sont implémentées — les modules métier (produits, ventes, finances, crédit, clients, achats) persistent par tenant et par magasin dans Supabase avec RLS + triggers anti-falsification et RPC d'argent atomiques.

Documents connexes

- Sécurité — stockage sécurisé et limites de responsabilité de la session.
- Domaine — IAuthRepository et les cas d'usage d'authentification.
- Présentation — AuthGate et LoginScreen.
- Multi-tenant et Super Admin — approbation des tenants, forfaits, magasins et employés.